Slide 1: ScholarPlan: from design to a working multi-agent system

Hello, my name is Adil Aljasmi. This presentation covers the second deliverable of the Intelligent Agents module: the implementation of ScholarPlan.

In Unit 6, our group designed ScholarPlan, an LLM-powered planning agent for academic research and information gathering. In Unit 11 I built that design, individually, as a working multi-agent system. Over the next twenty minutes I will show how the implementation meets the design, walk through it running on a real research question, explain how it was tested and evaluated against measurable targets, and reflect critically on what worked, what changed, and why.

Slide 2: From a research goal to verified, stored evidence

The problem ScholarPlan addresses is simple to state. Searching the literature is repetitive, but handing the task to a single large language model is risky: unaided LLMs plan unreliably (Kambhampati et al., 2024) and they fabricate citations and facts (Ji et al., 2023).

So ScholarPlan splits the work between five cooperating agents. You give it a research goal in plain English. The Planner decomposes it into search subtasks. The Retriever queries three open scholarly APIs: arXiv, OpenAlex and Crossref. The Processor removes duplicates, ranks the results and turns each abstract into short, atomic claims. The Critic, a separate model, checks every claim against the source it came from. Finally, the Archivist stores a cited report, a BibTeX file, a searchable vector index and a full reasoning trace offline.

This maps directly onto the three success criteria in the brief: relevant data is identified and retrieved; it is processed by filtering, summarising and verifying; and it is stored in a usable format.

The numbers on the right summarise where the project ended up. All five agents are implemented. There are 96 automated tests with 92 percent line coverage. And in live evaluation, three out of three research goals completed within the step budget, averaging under seven reasoning steps and about one minute each.

Slide 3: Hybrid agents coordinating through a blackboard

This slide shows the architecture, which follows the Unit 6 design closely.

ScholarPlan is a hybrid architecture (Wooldridge, 2009). There is one deliberative agent, the Planner, which reasons about the goal using an LLM. The other four agents are reactive specialists: each one simply waits for its input to appear and then acts. The Retriever and the Archivist use no LLM at all. That is deliberate: it keeps retrieval failures and reasoning

failures on separate surfaces, so an irrelevant search result is never confused with an unsupported claim (Guo et al., 2024).

The agents never call each other. Instead, they cooperate through a shared blackboard, the classic architecture described by Hayes-Roth (1985), which Salemi and colleagues (2025) have recently shown outperforming a central orchestrator for LLM agents doing information discovery. I implemented the blackboard in SQLite, so every plan, source, claim, verdict and piece of feedback persisted and can be inspected after the run. That is how the system addresses explainability.

Two implementation details matter here. First, coordination safety: when an agent takes a subtask, it does so inside a single immediate transaction, so two agents can never claim the same work. A unit test races eight threads for one subtask, and exactly one wins.

Second, I applied the Gaia methodology (Wooldridge, Jennings and Kinny, 2000): each agent is a role with explicit permissions over the blackboard tables. These permissions are enforced at runtime, not just documented. If the Retriever tried to write a verdict, the blackboard would raise a permission error.

Scheduling is handled by a tiny controller that asks each agent, "can you act?" and runs the first that can, finishing downstream work before the Planner deliberates again.

Slide 4: Planning with ReAct, Reflexion and LLM-Modulo guardrails

The Planner runs a ReAct loop (Yao et al., 2023). Each step has three parts. The observation is the current state of the blackboard: which subtasks are verified or rejected, what feedback has been written, and how much budget is left. The thought is the LLM's written reasoning, saved to the trace. The action is exactly one move: issue a subtask, refine its query, switch to a different database, abandon it, or synthesise the final answer.

Re-planning follows the Reflexion idea of verbal feedback (Shinn et al., 2023). When the Retriever finds nothing, or the Processor finds nothing relevant, or the Critic rejects the claims, that agent writes a natural-language feedback message. The Planner must read it before its next thought.

The most important design principle comes from Kambhampati and colleagues (2024): LLMs can't plan reliably on their own, but they can help planning inside an LLM-Modulo framework, where external components verify. So in ScholarPlan, the LLM never decides what is allowed. The code computes the allowed actions for each step. The LLM's output must pass a JSON schema, and then a semantic check: for example, it cannot refine a subtask with a query that has already failed. If the model fails twice, a deterministic fallback policy takes over.

Termination is guaranteed without trusting the model. There is a twenty-step ReAct budget, and, responding to the tutor's feedback on Unit 6, a hard cap of three re-plans per subtask. On the fourth failure the subtask is marked "unresolvable" by code, without even calling the LLM. A synthesis that the Critic rejects twice is replaced by an extractive list of verified claims, which by construction cannot contain unsupported text.

Slide 5: Structural view: UML class diagram

The Unit 6 feedback noted that our design had behavioural diagrams, an activity diagram and a sequence diagram, but no static structural view. This class diagram adds that view, and it is drawn from the actual code.

At the top, the Controller aggregates five Agent objects. Agent is an abstract class with two methods: can-act, a cheap side-effect-free trigger check, and act. The five roles inherit from it. Every agent reaches the blackboard only through a ScopedBlackboard, which enforces that role's permissions.

At the bottom left is the dependency-inversion principle at work. The three LLM-using agents depend on an LLMClient interface, not on Groq. There are three implementations: the live OpenAI-compatible client, a deterministic mock and a scripted mock used to force failure paths in tests. Switching provider is a configuration change. That is exactly what made the model changes I will discuss later cheap to make.

At the bottom right, the Retriever uses three API clients sharing one polite, cached, rate-limited HTTP component, and all thresholds live in a single frozen Settings object, so the numbers I quote are defined in one place.

Slide 6: Demonstration: live runs, step by step

These four panels are captured from real live runs, using the ScholarPlan dashboard. The dashboard reads the SQLite blackboard every second, so what you see is exactly what the agents recorded. The goal for this run was "How do LLM-based critics or verifiers detect hallucinated claims?". I also switched on a clearly labelled demonstration option that plants one false claim, because live runs rarely hallucinate on cue and I wanted to show the Critic catching one.

[Panel 1]

Panel one is the ReAct loop in action. The Planner's first thought explains how it will decompose the goal: detection methods, training signals, evaluation benchmarks and known limitations. Its action creates a four-subtask plan. At the next step it reasons that it needs evidence first, and issues the first of those subtasks, number five on this shared blackboard. From there the specialists take over through the blackboard. The run's saved

summary records that the Retriever collected sixty records from the three APIs, the Processor removed ten duplicates and kept the most relevant sources, and all four subtasks were verified in six of the twenty allowed steps.

[Panel 2]

Panel two is the key moment. Claim C22, the planted one, says the study "eliminates hallucination entirely and improves accuracy by 99 percent on every benchmark". The Critic, a model from a different family than the one that wrote the claims, rejected it, and its reason is precise: the source reports an AUROC of 0.781 and specific percentage improvements, not the elimination of hallucination. The genuine claims from the same abstract, C20 and C21, were accepted. Sixteen of seventeen claims survived, and the false one never reaches the report.

[Panel 3]

Panel three comes from an earlier live evaluation run, where re-planning happened for real. The Processor's model twice failed to return valid output, and each time it wrote feedback to the blackboard. The Planner read that feedback and re-planned twice, and the run still finished with all four subtasks verified, in eight steps. This is also where I found the misdiagnosis problem I will return to under testing.

[Panel 4]

Panel four is the stored result: the Archivist's report, shown after a citation-formatting fix I made while preparing these slides, remediation R20. The content is unchanged, and both versions are in the evidence folder. The report answers the goal with Harvard in-text citations, and each citation keeps its claim identifier, so every cited statement can be traced back to a verified claim and its source. The same run saved a BibTeX file, a searchable vector index and the full trace.

In the code, the logic behind these panels lives in a few places: the ReAct step and its validation in planner.py, the entailment check in critic.py, and the atomic subtask-claiming transaction, claim_subtask, in blackboard.py.

All of the evidence that the system works is in my public GitHub repository, shown at the bottom of the slide. The evidence/runs folder holds the terminal logs, reports, BibTeX files and traces of the live runs, including this one. The evidence/screenshots folder has ten dashboard screenshots, evidence/tests has the full test-suite output with coverage, evidence/evaluation has the benchmark results, and docs/REMEDIATION_LOG.md records every defect found and fixed. Section nine of the README maps each piece of evidence to its file, and the README explains how to run the system yourself.

Slide 7: Does it work? Evaluation against measurable targets

The Unit 6 feedback asked for clear, measurable targets: test queries, deduplication thresholds, and Critic precision and recall. I set these targets before running anything.

First, the Critic. I hand-labelled 21 claim-source pairs: twelve supported, and nine hallucinations of seven kinds, including over-generalisation, wrong entities, fabricated numbers, metric shifts and contradictions. The targets were precision of at least 0.85 and recall of at least 0.75. The chart compares three verifiers. A simple word-overlap baseline reaches only 0.83 and misses contradictions and metric shifts, because those reuse the source's own words. The production Critic, Qwen, and the fallback, gpt-oss-120b, both scored 1.0 on precision and recall, catching all nine hallucinations. That comparison is the empirical justification for using an LLM as the judge rather than string matching. I'll be honest about the limit, though: 21 items is small, and a perfect score is more likely a ceiling effect than proof of perfection.

Second, retrieval. I used ten fixed queries whose target papers are the ones our own Unit 6 report cited, in effect asking whether ScholarPlan can find the literature its design relied on. Recall was 0.9 against a target of 0.75. No single API found everything: arXiv found eight, OpenAlex six and Crossref one, which supports the decision to query all three. The one miss, Hayes-Roth 1985, has no indexed abstract, and the system deliberately excludes records it cannot verify. That is a real trade-off: verifiability versus coverage of older work.

Third, end-to-end. Three fixed goals, live: all three completed within the twenty-step budget, using 6.7 steps and under a minute on average, and about 21,000 tokens per run, comfortably inside the free tier.

The deduplication and relevance thresholds, 0.92 and 0.30 cosine similarity, are named constants and are unit-tested in both directions.

Slide 8: Testing, remediation and development progress

Testing followed the Unit 6 plan of testing by agent type. Deterministic components have conventional unit tests: 84 of them, covering blackboard atomicity and permissions, API parsers run against recorded real responses, rate limiting and retries, the thresholds, Planner rules and citation formatting. Twelve functional tests run the whole system end to end, offline, and each one forces a specific behaviour: Critic rejection leading to re-planning, the three-re-plan cap, budget exhaustion, an injected hallucination being filtered, and an API failing mid-run. Four live smoke tests detect drift in the external services. Overall: 96 passing tests and 92 percent line coverage.

The scripted mock LLM is what makes this possible: a real model will not reject claims on demand, but a scripted one can, deterministically.

Every defect is recorded in a remediation log, twenty in total, each with its root cause, fix and verifying test. Three stand out. The Critic model chosen in Unit 6 was withdrawn by the provider, so I added an automatic model-substitution mechanism and a regression test. OpenAlex returned "too many requests" errors in the first live run, so the client now honours the server's retry-after header. And most interesting: in a live run, the Planner misdiagnosed a model formatting error as a bad query and rewrote a perfectly good query, because the feedback message did not state the cause. The lesson is that Reflexion-style self-correction is only as good as its feedback signal, so feedback now carries the diagnosis.

Progress is visible in the Git history: from the Unit 6 design, through eight short sprints, Sprint 0 to Sprint 7, each ending in a tested increment.

Slide 9: Critical reflection: deviations, limitations and ethics

Three deviations from the design are worth defending. First, no Azure subscription was available, so the Planner and Processor run OpenAI's open-weight gpt-oss models on Groq's free tier instead of GPT-5-mini on Azure. Because the client speaks the OpenAI-compatible API, moving back to Azure is a one-line change. Second, Groq no longer offers Llama-3.3-70B to new accounts, so the Critic is Qwen. What the design actually needed was independence: a verifier from a different model family from the generator, and Qwen preserves that. Third, without a GPU, the local seven-billion-parameter checker was replaced in testing by deterministic mocks, which are better at forcing failure paths anyway.

The main limitations are these. Verification is at abstract level only. The Critic can be lenient on connecting prose: in one live run it accepted an uncited bridging sentence. A sentence-level citation rule would close that gap. And the Critic benchmark is small.

On legal, social, ethical and professional factors: the system respects each API's terms, including arXiv's one request every three seconds, identifies itself, and caches responses. It stores only metadata and abstracts, never full texts, and attributes every claim. Rejected claims never reach the report, and the report states that it is AI-generated and verified at abstract level, so it is a starting point for reading, not a substitute for it. Bias exists, towards English and recent work, and is stated openly. The research goal is sent to a third-party provider, so users should not enter personal data. API keys are kept out of version control, and traces are scrubbed of local file paths.

Slide 10: Conclusion

To conclude: ScholarPlan implements the Unit 6 design as a working, tested multi-agent system. A deliberative ReAct planner and reactive specialists coordinate through a blackboard, an independent Critic filters hallucinations, and the results are stored offline

with full provenance. It meets the measurable targets the design was criticised for lacking, and every change from the plan is documented and justified.

The broader lesson is the LLM-Modulo one: let the language model supply judgement and let code guarantee validity and termination.

The code, tests, evidence and README with full references are in my GitHub repository. Thank you for listening.

References cited in the transcript

Guo, T. et al. (2024) 'Large language model based multi-agents: a survey of progress and challenges', in *Proceedings of IJCAI-24*, pp. 8048–8057. Available at: https://doi.org/10.24963/ijcai.2024/890 (Accessed: 11 October 2026).

Hayes-Roth, B. (1985) 'A blackboard architecture for control', *Artificial Intelligence*, 26(3), pp. 251–321. Available at: https://doi.org/10.1016/0004-3702(85)90063-3 (Accessed: 11 October 2026).

Ji, Z. et al. (2023) 'Survey of hallucination in natural language generation', *ACM Computing Surveys*, 55(12), pp. 1–38. Available at: https://doi.org/10.1145/3571730 (Accessed: 11 October 2026).

Kambhampati, S. et al. (2024) 'Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks', in *Proceedings of ICML 2024*. Vienna: PMLR 235. Available at: https://arxiv.org/abs/2402.01817 (Accessed: 11 October 2026).

Salemi, A. et al. (2025) 'LLM-based multi-agent blackboard system for information discovery in data science', *arXiv preprint* arXiv:2510.01285. Available at: https://arxiv.org/abs/2510.01285 (Accessed: 11 October 2026).

Shinn, N. et al. (2023) 'Reflexion: language agents with verbal reinforcement learning', in *Advances in Neural Information Processing Systems 36*. Available at: https://arxiv.org/abs/2303.11366 (Accessed: 11 October 2026).

Wooldridge, M. (2009) *An Introduction to MultiAgent Systems*. 2nd edn. Chichester: John Wiley & Sons.

Wooldridge, M., Jennings, N.R. and Kinny, D. (2000) 'The Gaia methodology for agent-oriented analysis and design', *Autonomous Agents and Multi-Agent Systems*, 3(3), pp. 285–312. Available at: https://doi.org/10.1023/A:1010071910869 (Accessed: 11 October 2026).

Yao, S. et al. (2023) 'ReAct: synergizing reasoning and acting in language models', in *Proceedings of ICLR 2023*. Available at: https://arxiv.org/abs/2210.03629 (Accessed: 11 October 2026).